

Week_4 Task 1: Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

```
GNU nano 8.7.1 wait_waitp:
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    int i;
    pid_t pid;
    int status;
    printf("Parent Process PID: %d\n", getpid());
    for (i = 1; i <= 3; i++)
    {
        pid = fork();
        if (pid < 0)
        {
            perror("fork failed");
            return 1;
        }
        if (pid == 0)
        {
            printf("Child %d: PID = %d, Parent PID = %d\n", i, getpid(), getppid());
            sleep(i);
            printf("Child %d completed.\n", i);
            exit(i);
        }
    }
    printf("\nParent is waiting using wait()...\n");
    pid = wait(&status);
    if (pid > 0)
    {
        printf("wait(): Child with PID %d completed.\n", pid);
    }
    printf("\nParent is waiting using waitpid()...\n");
    for (i = 0; i < 2; i++)
    {
        pid = waitpid(-1, &status, 0);
        if (pid > 0)
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

OUTPUT:

```
bash: ./multiple_children: No such file or directory
the_boss@DESKTOP-I8C8AJG:~/ossp$ ./multiple_children
Parent Process PID: 1053
Child 1: PID = 1054, Parent PID = 1053
Child 2: PID = 1055, Parent PID = 1053

Parent is waiting using wait()...
Child 3: PID = 1056, Parent PID = 1053
Child 1 completed.
wait(): Child with PID 1054 completed.

Parent is waiting using waitpid()...
Child 2 completed.
waitpid(): Child with PID 1055 completed.
Child 3 completed.
waitpid(): Child with PID 1056 completed.

All child processes completed.
```

REPORT: I created a parent process that creates multiple child processes using `fork()`. I used `wait()` to wait for a child process to complete and used `waitpid()` to synchronize the remaining child processes. From the output, I understood that `wait()` can wait for any child process, while `waitpid()` gives more control over which child process is waited for

Task 2: Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.



the_boss@DESKTOP-I8C8AJG: ×



GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0)
    {
        printf("Child: PID = %d\n", getpid());
        printf("Child is exiting now...\n");
        exit(0);
    }
    else
    {
        printf("Parent: PID = %d\n", getpid());
        printf("Parent is sleeping...\n");

        // Parent does NOT call wait()
        sleep(30);

        printf("Parent finished sleeping.\n");
    }

    return 0;
}
```

OUTPUT:

```
the_boss@DESKTOP-I8C8AJG:~/oss$ ./zombie
Parent: PID = 1104
Parent is sleeping...
Child: PID = 1105
Child is exiting now...
Parent finished sleeping.
```

GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0)
    {
        printf("Child: PID = %d\n", getpid());
        printf("Child is exiting...\n");
        exit(0);
    }
    else
    {
        printf("Parent: PID = %d\n", getpid());
        printf("Parent waiting for child...\n");

        wait(&status);

        if (WIFEXITED(status))
        {
            printf("Parent: Child completed successfully.\n");
        }
    }
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^ Replace

^K Cut
^U Paste

OUTPUT:

```
the_boss@DESKTOP-I8C8AJG:~/oosp$ ./zombie
Parent: PID = 1166
Parent waiting for child...
Child: PID = 1167
Child is exiting...
Parent: Child completed successfully.
Parent process completed.
```

REPORT: I created a child process that becomes a zombie when the parent does not immediately collect its exit status. I checked the process table and observed the zombie process. Then I modified the program by using proper synchronization with `wait()` so that the parent collects the child process after it finishes. I understood that proper synchronization helps to prevent zombie processes.